Agent Development Kit (ADK) by Michaël Bettan

Definition: Agent Development Kit (ADK) is Google's open-source, code-first framework (**Python, TypeScript, Go, Java, and Kotlin**) for building, evaluating, governing, and deploying production AI agents. Rather than leaving teams to hand-roll conversation state, tool dispatch, retries, and trace instrumentation around a raw model endpoint, ADK provides the agent harness as modular infrastructure: an event-sourced execution runtime, typed tool and protocol connectors, multi-tier state and memory, multi-agent orchestration, deterministic callback/plugin guardrails, trajectory evaluation, and one-command cloud deployment.

Why ADK? (What a raw model API lacks): A bare model endpoint is **stateless** (retains zero memory across HTTP requests unless using server-side interaction handles), **passive** (emits tool-call intents as JSON text without executing anything), **unstructured in control flow** (no durable loops, fan-out joins, checkpoints, or hard iteration caps), and **ungoverned** (no tenant isolation, approval gates, or tool-boundary PII scrubbing). ADK supplies all four layers cleanly decoupled from model weights.

> MENTAL MODEL
> Treat the foundation model (**Gemini 4 Argon**, **Gemini 3.8 Flash**, **Claude**, or **Gemma 4**) as the **brain**, and ADK as the **nervous system and skeleton**: hands (`FunctionTool` , `McpToolset`), short- and long-term memory (`SessionService` , `MemoryService`), spinal reflexes (`before_tool_callback` , `BasePlugin`), and the motor loop (`Runner` / `Workflow`) that coordinates when the brain reasons and when tools act.

End-to-end lifecycle: 1. Define (`LlmAgent` / `Workflow` / `ManagedAgent` + tools + instructions) → **2. Run** (`Runner` loads session state, invokes the model or **Interactions API**, executes tools, and commits each step as an immutable `Event`) → **3. Evaluate** (`adk conformance` + `adk eval` in CI, paired with **Agent Platform Gen AI Evaluation Service** in cloud) → **4. Deploy** (**Agent Runtime**, **Cloud Run**, or **GKE**).

1. When to use ADK, framework choices and multi-language parity

> RULE 0
> Begin with a single grounded model call. Step up to deterministic workflows—and only then to autonomous multi-turn agents—**when benchmark evaluations prove the simpler tier cannot meet accuracy or adaptability targets.**

Architectural tier	Core mechanism	Makes runtime routing decisions?	Executes external actions?
Single LLM call	Stateless prompt → completion	✅ (single step)	❌ (can only emit tool-call JSON)
Deterministic workflow	Predefined DAG or pipeline orchestrating models and tools	❌ (code/edges decide the path)	✅
Autonomous agent	Model-directed loop executing tools, inspecting observations, and updating state until goal completion	✅ (multi-step adaptive)	✅
ADK	Production framework providing both deterministic workflows and autonomous agent loops under one event ledger	n/a	n/a

Workload shape	Recommended pattern	Architectural rationale
Single prompt (+ retrieval) answers reliably	Augmented LLM / RAG call	Adding an agent loop inflates TTFT, token spend, and variance for zero gain
Fixed, predictable multi-step pipeline (e.g., parse → validate → summarize)	**ADK 2.0** `Workflow` **graph** (or ADK 1.x `SequentialAgent` / `ParallelAgent` / `LoopAgent`)	Deterministic execution order, bounded cost, straightforward debugging
Read-only Q&A over enterprise SharePoint/Drive corpora with ACLs	Managed **Agent Search** / RAG pipeline	Retrieval grounding solves missing knowledge without open-ended tool loops
Next action depends dynamically on intermediate tool findings	`LlmAgent` (ReAct / planner loop)	Control path cannot be hardcoded ahead of time
Long-horizon cloud sandbox execution using Google's hosted harness	`ManagedAgent` (backed by **Interactions API**)	Offloads loop, remote container sandbox, and server-side state to Google
Strictly deterministic compliance or actuarial calculation	Plain code / classical ML	Zero stochastic variance, 100% reproducible auditability

The "agent theater" litmus test: Splitting a system into multiple agents is justified **only when specialists require distinct tool permissions, isolated data scopes, an independent auditor (`include_contents='none'` + verification tools), or explicit human sign-off boundaries.** Breaking a single prompt into four chatty persona agents sharing identical context and zero tools burns ~4× the tokens and ~2× the latency without improving accuracy.

SDK language	Package / module	ADK 2.0 graph `Workflow` & task modes	Primary strengths & parity nuances (Oct 2026)	Typical Google Cloud deploy target
Python	`google-adk`	✅ Full support	Reference flagship SDK: earliest access to `ManagedAgent` , `SkillToolset` + `GCPSkillRegistry` , `adk eval` / `AgentEvaluator` / `adk conformance` , `VertexAiMemoryBankService` , and `run_live()` . Agent Runtime upload excludes the API server (platform injects it).	**Agent Runtime**, Cloud Run, GKE

SDK language	Package / module	ADK 2.0 graph `Workflow` & task modes	Primary strengths & parity nuances (Oct 2026)	Typical Google Cloud deploy target
TypeScript	`@google/adk`	✅ Full support	Node.js / full-stack web integration, MCP servers, experimental `RoutedLlm` dynamic model switcher, and `BuiltInCodeExecutor` without single-tool mixing limits on modern Gemini models (`Agent Search` built-in tool not yet in TS).	**Cloud Run**, GKE
Go	`google.golang.org/adk`	✅ Full support	High-concurrency, low-memory compiled microservices with native goroutine fan-out, MCP, OpenTelemetry, and **Agent Runtime** support (Go deploy payload bundles its own dedicated ADK API server binary).	**Agent Runtime**, Cloud Run, GKE
Java & Kotlin	`com.google.adk`	❌ (ADK 1.x workflow agents)	Enterprise Spring/JVM integration (`LlmAgent.builder()` in Java; type-safe DSL in Kotlin) with `SequentialAgent` / `ParallelAgent` / `LoopAgent` , `FunctionTool` , `McpToolset` , and callbacks; built-in `GoogleSearchTool` + custom tools must be isolated via `AgentTool` .	**Cloud Run**, GKE

COMMON TRAP

Confusing the **Gen AI SDK (`google-genai`)** with **ADK (`google-adk`)**. `google-genai` is the model/interactions client library (supporting `generate_content` , `client.interactions.create` , and basic in-process function calling), whereas ADK provides multi-user `Session Service` isolation, the persistent `Event` ledger, multi-agent transfers, lifecycle guardrails (`BasePlugin`), conformance/trajectory evals, and runtime deployment.

2. Runtime architecture: Agent, Runner, SessionService and ManagedAgent

ADK enforces a strict separation between **stateless agent definitions**, **stateful session storage**, and the **event-loop engine**:



Core primitive	Responsibility	Production engineering rule
`Agent` (`Llm Agent`)	Stateless blueprint: model, instructions, tools, sub-agents, planners, callbacks	Store **zero** per-user attributes on `self` ; one singleton safely handles thousands of concurrent requests
`Managed Agent`	Data-plane client for Google's **Managed Agents API** via the **Interactions API**	Connects to an existing `agent_id` (`global` endpoint on Agent Platform); runs loop & tools remotely
`BaseLlm`	Model abstraction (`"gemini-4-argon"` , `"gemini-3.8-flash"` , `LiteLlm(...)` , `Claude(...)` , `Gemma(...)`)	Mix model families across coordinator and worker agents without changing session or callback code
`Runner`	Executes turns (`run_async`) or bidirectional audio/video (`run_live`)	**Commit-then-resume:** persists every `Event` to `SessionService` before advancing to the next step
`Session Service`	Manages `Session` objects keyed by `app_name` / `user_id` / `session_id`	`InMemorySessionService` (local dev) → `DatabaseSessionService` (`postgresql+asyncpg` / `sqlite+aiosqlite`) or `VertexAi SessionService` (prod; `app_name` = Reasoning Engine ID)
`Memory Service`	Cross-session long-term recall per user	`InMemoryMemoryService` (unit tests) → `VertexAiMemoryBank Service` (consolidated facts) / `VertexAiRagMemoryService`

Core primitive	Responsibility	Production engineering rule
`Artifact Service`	Versioned binary/file blobs (`"user:filename"` persists across sessions)	`InMemoryArtifactService` / `File ArtifactService` (dev) → `Gcs ArtifactService` (Cloud Storage in prod)
`App`	Root application envelope binding `root_agent` , `plugins=[...]` , and runtime policies	Attach `ResumabilityConfig` , `EventsCompactionConfig` , and `ContextCacheConfig` here—never on `LlmAgent`

`LlmAgent` (`GenerateContent`) vs. `ManagedAgent` (`Interactions API`): - **Standard `LlmAgent` :** Your ADK `Runner` owns the orchestration loop, re-sending compacted conversation history (and cryptographic **Thought Signatures**) over `GenerateContent` while executing custom Python `FunctionTool` s and `McpToolset` connections inside your runtime environment. - **`ManagedAgent` (Preview):** Invokes a pre-existing hosted agent (`agent_id` , such as `antigravity-preview-...` or a custom agent provisioned via the **Agents API** control plane) over the **Interactions API** (`stream=True`) with `environment={'type': 'remote'}` . Reasoning, built-in tools (`google_search` , `ToolCodeExecution`), and sandbox execution happen inside Google's managed cloud infrastructure while partial and final events stream into your ADK workflow. - **State split in `Managed Agent` :** ADK never re-transmits prior conversation turns to a `Managed Agent` . Instead, the local `Session` records only two lightweight pointers on emitted events— **`previous_interaction_id`** and **`environment_id`** —and reuses them on subsequent turns so the server-side `Interaction` and

remote container sandbox continue seamlessly. - **Direct `Interactions API` (`client.interactions.create`) for long-horizon tasks:** When calling `gemini-4-argon` or Deep Research directly via `google-genai`, setting `background=True` runs multi-minute tasks asynchronously on the server (preventing ~60s HTTP gateway timeouts and unlocking long decodes up to the unified `1,048,576`-token window). Chaining a follow-up request via `previous_interaction_id` while the prior interaction is still `in_progress` returns `400 Bad Request`—poll until `completed` first.

```
import os
from google.adk.agents import LlmAgent, ManagedAgent
from google.adk.tools import google_search
from google.adk.tools.agent_tool import AgentTool

# ManagedAgent runs server-side via the Interactions API (global endpoint
on Agent Platform)
web_researcher = ManagedAgent(
    name="managed_web_researcher",
    description="Performs grounded web synthesis in Google's remote
managed environment.",
    agent_id=os.environ.get("MANAGED_AGENT_ID", "antigravity-preview-09-
2026"),
    environment={"type": "remote"},
    tools=[google_search],  # Server-side tools only; local FunctionTools
raise NotImplementedError
)

# Coordinator combines local/MCP tools with the remote ManagedAgent
wrapped as an AgentTool
coordinator = LlmAgent(
    name="portfolio_coordinator",
    model="gemini-4-argon",
    instruction="Combine internal CRM lookups with external web
research.",
    tools=[AgentTool(agent=web_researcher)],
)
```

Context hierarchy & Event ledger: `InvocationContext` (full per-turn runtime handle), `ReadonlyContext` (passed to `InstructionProvider` callables for read-only `ctx.state`), `CallbackContext` (agent/model hooks: mutable state + artifacts), and `ToolContext` (tools + tool hooks: state, artifacts, credentials, `search_memory`, confirmation, and `actions`). Every state change, tool call, transfer, or escalation is recorded as an `Event` (`author`, `invocation_id`, `content`, `actions.state_delta`, `actions.artifact_delta`), creating a single source of truth for `adk web` time-travel inspection, Cloud Audit trails, and trajectory grading.

> **COMMON TRAP**
> Passing client-executed Python functions or an `McpToolset` into `ManagedAgent(tools=[...])` raises **`NotImplementedError`**, and pointing `ManagedAgent` at a regional Agent Platform endpoint fails because the Managed Agents API is served exclusively from **`global`**. Keep custom/MCP tools on an `LlmAgent` and compose `ManagedAgent` beside it.

3. Agent types and deterministic control flow

Every ADK agent inherits from `BaseAgent`. Only `LlmAgent` (and `ManagedAgent`) consult a model to decide next steps; **workflow agents route deterministically in code with zero LLM routing overhead.** *(In ADK 2.0 for Python, TypeScript, and Go, `SequentialAgent`, `ParallelAgent`, `LoopAgent`, and custom `BaseAgent` orchestrators are superseded by unified graph `Workflow`s in §5, though both patterns remain active in Java/Kotlin and exam scenarios.)*

Agent class	Routing mechanism	Ideal workload	Critical configuration
`LlmAgent` (`Agent`)	Dynamic model reasoning	Tool selection, open-ended synthesis, triage	`model`, `output_key`, `output_schema`, `planner`, `tools`, `sub_agents`
`SequentialAgent`	Strict linear order `A → B → C`	Staged pipelines where step `N` consumes step `N-1`'s `output_key`	`sub_agents=[...]`
`ParallelAgent`	Concurrent execution (`branch` isolated, shared `session.state`)	Multi-source retrieval, independent compliance checks	Unique `output_key` per parallel branch
`LoopAgent`	Iterative cycle until exit signal or cap	Draft → critique → revise loops, polling	`max_iterations` + tool setting `actions.escalate = True`
Custom (`BaseAgent`)	Arbitrary async code	Deterministic rule routers, custom state machines	Override `async def _run_async_impl(self, ctx)`

```
from google.adk.agents import LlmAgent, SequentialAgent, ParallelAgent,
LoopAgent
from google.adk.tools import ToolContext

def approve_patch(tool_context: ToolContext) -> dict:
    """Invoke ONLY when static analysis and unit tests report zero
defects."""
    tool_context.actions.escalate = True            # Halts the enclosing
LoopAgent immediately
    tool_context.actions.skip_summarization = True  # Avoids redundant
post-tool LLM turn
    tool_context.state["temp:verified"] = True      # Proves convergence
vs. hitting max_iterations
    return {"status": "verified"}

gather = ParallelAgent(name="gather_context", sub_agents=[
    LlmAgent(name="cve_scan", model="gemini-3.5-flash-lite",
instruction="Extract CVE specs.", tools=[read_advisory],
output_key="cve"),
    LlmAgent(name="ast_scan", model="gemini-3.5-flash-lite",
instruction="Locate affected symbols.", tools=[search_repo],
output_key="symbols"),
])
patch_loop = LoopAgent(name="patch_and_verify", max_iterations=3,
sub_agents=[
    LlmAgent(name="patcher", model="gemini-3.8-flash", instruction="Write
patch for {cve} in {symbols}. Feedback: {audit?}", output_key="diff"),
    LlmAgent(name="auditor", model="gemini-4-argon",
include_contents="none",
             instruction="Verify {diff} with test tools. Call
approve_patch only if all checks pass.",
             tools=[run_tests, approve_patch], output_key="audit"),
])
root_agent = SequentialAgent(name="remediation_pipeline", sub_agents=
[gather, patch_loop])
```

- **State templating & `InstructionProvider`:** Setting `output_key="cve"` writes the agent's final text to `state["cve"]`, which downstream instructions reference via `{cve}` (use `{audit?}` when optional so turn 1 doesn't raise `KeyError`, or `{artifact.report}` to inline text artifacts). Whenever an instruction includes literal curly braces—such as JSON schemas `{"status": "ok"}` or regex quantifiers—supply a callable `def build_instruction(ctx: ReadonlyContext) -> str:` so ADK bypasses `{var}` placeholder parsing.
- **Silent `LoopAgent` exhaustion:** Reaching `max_iterations` exits `LoopAgent` **silently without raising an exception**, allowing any subsequent agent in an outer `SequentialAgent` to run on unapproved output. Always write an explicit convergence flag (`state["temp:verified"] = True`) inside your exit tool and check it in the downstream step's `before_agent_callback`.
- **Independent verification:** Prevent self-grading bias in critic loops by isolating the verifier (`include_contents="none"`, dedicated deterministic test/linter tools, and explicit pass/fail criteria).

> **COMMON TRAP**
> Expecting `SequentialAgent` to skip irrelevant steps dynamically or expecting `LoopAgent(max_iterations=3)` to stop automatically when tests pass. `SequentialAgent` always runs every child in sequence, and `LoopAgent` always runs all `max_iterations` unless a tool or callback sets `escalate = True`.

4. Multi-agent delegation: transfer, AgentTool and remote agents

Delegation pattern	Invocation mechanics	Who speaks to the user next?	Session & context boundary	Best architectural fit
`sub_agents` (LLM transfer)	ADK injects `transfer_to_agent(agent_name=...)`; LLM selects target using each child's **`description`**	**Target sub-agent** assumes control	Shares the same `session.events` and `session.state`	Full conversational handoff to a domain specialist (billing vs. tech support)
`AgentTool(agent=...)`	Parent invokes child like a typed function (`skip_summarization=True` returns raw output)	**Parent coordinator** keeps control	Runs in an isolated child session while **merging `state_delta` / `artifact_delta` (`output_key`) back**	Modular sub-task execution without polluting the coordinator's chat history
`RemoteA2aAgent`	Connects to an external service via its A2A Agent Card (`/.well-known/agent-card.json`)	Depends on transfer vs. `AgentTool` wrapping	Strictly opaque network boundary: only A2A messages, tasks, and artifacts cross	Cross-team, multi-language, or cross-organization agent federation

- **Scoping transfers & tree rules:** Use `disallow_transfer_to_parent=True` on one-shot triage helpers (blocks upward handoff and routes the user's next turn back to `root_agent`) and `disallow_transfer_to_peers=True` to enforce strict hub-and-spoke governance through a coordinator. Every `Agent` instance obeys the **single-parent rule** (`ValueError` if attached to two parents' `sub_agents`); use factory functions or `agent.clone()` to reuse definitions.
- **ADK 2.0 collaborative sub-agent modes (Python, TS, Go):**
- `mode="chat"` (classic default): injects one shared `transfer_to_agent` tool; the specialist takes over user interaction.
- `mode="task"`: specialist can ask the user clarifying questions across turns, then invokes `finish_task` to hand structured output back to the parent coordinator.
- `mode="single_turn"`: generates a dedicated delegation tool per specialist, runs autonomously with zero direct user prompts, and supports parallel fan-out calls from the coordinator.
- **Cost-efficient routing:** Replace frontier LLM routers with deterministic `Workflow` conditional edges (`Event(route=...)`), embedding similarity, or **`gemini-3.5-flash-lite`** (`thinking_level="minimal"`) when intent buckets are well-defined.

> **COMMON TRAP**
> Writing instructions like `"Call transfer_to_billing()"` in `mode="chat"`. ADK does **not** create per-agent transfer functions in chat mode—it exposes a single `transfer_to_agent(agent_name="billing")` tool (per-specialist tools are generated only in ADK 2.0 `mode="single_turn"` / `"task"` or via `AgentTool`).

5. Graph workflows (ADK 2.0), planners and Gemini 4 Argon thinking levels

ADK 2.0 Graph Engine (`Workflow`): Available across **Python, TypeScript, and Go**, `Workflow(edges=[...])` unifies sequential pipelines, parallel fan-out/fan-in, cyclic loops, and conditional routing into one directed graph combining `LlmAgent`s, plain code nodes (`Function`, `Node`), `ManagedAgent`s, and nested `Workflow`s. Return values pass as typed `node_input`, and function parameters matching `session.state` keys are dependency-injected automatically.

```
from google.genai import types
from google.adk import Workflow
from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.adk.workflow import START, END, JoinNode
from google.adk.events import Event

def classify_severity(node_input: str) -> Event:
    target = "deep_architect" if "architecture" in node_input.lower()
else "fast_responder"
    return Event(output=node_input, route=target)  # Deterministic
conditional edge

deep_architect = LlmAgent(
    name="deep_architect",
    model="gemini-4-argon",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=True,
thinking_level="high")
    ),
    instruction="Produce a rigorous root-cause remediation plan.",
)
fast_responder = LlmAgent(
    name="fast_responder",
    model="gemini-3.8-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=False,
thinking_level="low")
    ),
    instruction="Resolve standard operational questions concisely.",
)

root_agent = Workflow(name="support_graph", edges=[
    (START, classify_severity),
    (classify_severity, {"deep_architect": "deep_architect",
"fast_responder": "fast_responder"}),
    (("deep_architect", "fast_responder"), END),
])
```

Graph capability	LangGraph equivalent	ADK 2.0 `Workflow` (Python / TS / Go)
Conditional branching	`StateGraph` + `add_conditional_edges`	`Workflow(edges=[...])` + node returning `Event(output=..., route="target")`

Graph capability	LangGraph equivalent	ADK 2.0 `Workflow` (Python / TS / Go)
Parallel fan-out & synchronization	Parallel edges + `Annotated[..., operator.add]` reducers	Edge tuple `(src, ("branch_a", "branch_b"))` + **`JoinNode`** (`dict[node_id, output]`); no state reducers—use distinct state keys
Durable checkpointing	Checkpointer keyed by `thread_id`	`SessionService` keyed by `app_name` / `user_id` / `session_id`
Mid-graph human pause	`interrupt()`	Generator node yielding `RequestInput(message=..., schema=...)` or `LongRunningFunctionTool`

Planners & October 2026 `thinking_level` Matrix (`PAA 3.1`): - **`BuiltInPlanner(thinking_config=types.ThinkingConfig(...))`** : Activates native Gemini System-2 reasoning (`part.thought == True` when `include_thoughts=True`) and preserves latent **Thought Signatures** across multi-turn tool loops. - **`PlanReActPlanner()`** : Prompts non-thinking or third-party models to emit explicit `/*PLANNING*/` → `/*ACTION*/` → `/*REASONING*/` → `/*FINAL_ANSWER*/` scaffolding tags. - **`thinking_level`** **(`low` | `medium` | `high` | `minimal`) vs. legacy `thinking_budget`** : `thinking_level` is **mandatory on Gemini 4 Argon** and **recommended across Gemini 3.x** (whereas integer `thinking_budget` is a legacy Gemini 2.5 parameter).

`thinking_level`	`gemini-4-argon`	`gemini-3.8-flash`	`gemini-3.5-flash-lite`	Architectural sweet spot
`"minimal"`	❌ **HTTP 400 Error**	❌ **HTTP 400 Error**	✅ **Supported (Default)**	Ultra-low-latency intent routers, schema extractors, inline classifiers
`"low"`	✅ Supported	✅ Supported	✅ Supported	Routine MCP tool execution, straightforward RAG synthesis, cost-sensitive turns
`"medium"`	✅ **Supported (Default)**	✅ **Supported (Default)**	✅ Supported	Balanced production default for multi-step agentic tool loops
`"high"`	✅ Supported (Dynamic)	✅ Supported (Dynamic)	✅ Supported (Dynamic)	Ambiguous planning, repository-scale code synthesis, conflict resolution (higher TTFT)

COMMON TRAP

Passing `thinking_level="minimal"` on **`gemini-4-argon`** or **`gemini-3.8-flash`** , OR passing legacy integer `thinking_budget=...` on **`gemini-4-argon`** , fails immediately with **HTTP `400 INVALID_ARGUMENT`** . Note also that `gemini-4-argon` ignores custom `temperature` , `top_p` , and `top_k` values—control reasoning depth and determinism via `thinking_level` , structured schemas, and explicit rubric instructions.

6. State, sessions and memory

Storage tier	Data shape	Lifecycle scope	ADK interface
Session events	Chronological immutable turn & tool transcript	Single conversation thread	`SessionService` (`session.events`)
Scoped state	Exact structured key/value scratchpad	Invocation, session, user, or app (by prefix)	`output_key` , `tool_context.state` , `EventActions(state_delta=...)`
Semantic memory	LLM-distilled, consolidated user facts retrieved by relevance	Cross-session per `user_id`	`MemoryService` (`VertexAiMemoryBankService`) + `load_memory` / `PreloadMemoryTool`
Artifacts	Versioned binary blobs (PDFs, images, CSVs; `"user:..."` = cross-session)	Session or user	`ArtifactService` (`GcsArtifactService` in prod)

State key prefix	Visibility scope	Survives process restart?
(unprefixed)	Current `session_id` only	Only with `DatabaseSessionService` or `VertexAiSessionService`
`user:`	All sessions belonging to `user_id` under `app_name`	Only with `DatabaseSessionService` or `VertexAiSessionService`
`app:`	Global across all users/sessions of `app_name`	Only with `DatabaseSessionService` or `VertexAiSessionService`
`temp:`	Current single invocation (shared across sub-agents in that turn)	**Never persisted** (stripped prior to commit)

- **Memory Bank vs. RAG Memory vs. Enterprise Grounding (`PAA 3.1 / 3.2`):**
- **`VertexAiMemoryBankService`** **(Managed Memory Bank):** Runs LLM fact extraction and **contradiction consolidation** (`await memory_service.add_session_to_memory(session)` → `GenerateMemories` , typically triggered in `after_agent_callback`). If a user changes preferences in Session 5, Memory Bank revises the stored memory rather than returning stale facts (`add_memory` / `CreateMemory` writes verbatim entries unless `enable_consolidation=True`).
- **`VertexAiRagMemoryService`** : Indexes raw session chunks into a RAG corpus (`similarity_top_k` , `vector_distance_threshold` where **smaller = closer**); does **not** reconcile conflicting historical statements.
- **Shared enterprise grounding (not per-user memory):** Use `VertexAiSearchTool` (**Agent Search** with native SharePoint/Drive ACL propagation), `VertexAiRagRetrieval` (**RAG Engine**), or `google_search` .
- **`load_memory`** **vs. `PreloadMemoryTool`** : `load_memory` lets the model query memory on demand (cheaper, risk of skipped recall); `PreloadMemoryTool` injects relevant memories automatically at the start of every turn (consistent personalization at higher per-turn latency/tokens).

COMMON TRAP

Mutating `session.state["user:plan"] = "pro"` directly on the object returned by `await session_service.get_session(...)` . That mutates a detached snapshot, skips `append_event()` , leaves zero audit trail, and drops the write in PostgreSQL / Agent Runtime. Always mutate state through `tool_context.state` , `callback_context.state` , `output_key` , or `EventActions(state_delta={...})` .

7. Tools and structured output

Passing a callable in `tools=[lookup_invoice]` wraps it as a `FunctionTool` whose **docstring** defines when the LLM invokes it and whose **type annotations** compile into the JSON Schema. Poor docstrings directly cause tool-selection hallucinations.

Tool design rule	Architectural reason	Implementation pattern
Atomic single responsibility	Monolithic tools hide unobservable logic and bloat failure blast radius	Separate `crm_get_customer` from `crm_issue_refund`
Namespace prefixes	Prevents collisions when combining multiple MCP servers and toolsets	`jira_create_issue`, `bq_execute_sql`
Document constraints & units	Models plan accurately when limits are explicit in the schema description	`"Read-only SQL; max 500 rows; 15 s timeout; dates in ISO-8601."`
Return structured errors, don't raise	Uncaught exceptions abort the turn; error dicts let the LLM self-correct or escalate	`{"status": "error", "error_code": "NOT_FOUND", "retryable": False}`
Enforce security inside tool code	Prompts cannot act as security boundaries against injection	Canonical path `resolve().is_relative_to(ROOT)` + `WriteMode.BLOCKED`
Idempotent mutations	`ResumabilityConfig` and transport retries have at-least-once semantics	Require `idempotency_key` before charging cards or creating tickets
Cap active tools (~16 max)	Tool selection precision degrades sharply when schemas crowd context	Partition across specialist `AgentTool`s or progressive `SkillToolset`s

- **Built-in tool mixing rules across SDK languages:** Gemini server-side tools (`google_search`, `VertexAiSearchTool`, `BuiltInCodeExecutor`) execute inside the model API call.
- In **ADK Python (v1.16+)**, set `bypass_multi_tools_limit=True` on `GoogleSearchTool` or `VertexAiSearchTool` to mix them alongside custom `FunctionTool`s.
- Combining **`BuiltInCodeExecutor`** with custom tools (in Python or TypeScript), OR combining **`GoogleSearchTool`** **with custom tools in Java/Kotlin**, is rejected unless you isolate each built-in tool inside its own specialist `LlmAgent` wrapped in **`AgentTool(agent=...)`**.
- **Google Cloud native toolsets:** `BigQueryToolset` (`BigQueryToolConfig(write_mode=WriteMode.BLOCKED)`), `SpannerToolset`, `Toolbox Toolset` (MCP Toolbox for Databases with connection pooling & IAM auth), `ApiHubToolset`, `ApplicationIntegrationToolset`, `OpenAPIToolset`, `McpToolset`, and `SkillToolset`.
- **Structured output (`output_schema`):** Pass a Pydantic model (`output_schema=TriageVerdict`) with `output_key="verdict"` for schema-guaranteed JSON. When `output_schema` conflicts with multi-turn tool calling on a given model/SDK configuration, decouple them in a 2-step `SequentialAgent` / `Workflow`: Step 1 runs tools and writes raw findings to state; Step 2 runs a stateless formatter (`output_schema=TriageVerdict`, `include_contents="none"`).

8. MCP, A2A and Agent Registry integration

MCP is vertical (`Agent → Tools/Resources`); A2A is horizontal (`Agent → Peer Agent`). Enterprise architectures combine both under **Agent Registry** governance (`PAA 3.2 / 3.3`).

```
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import
StreamableHTTPConnectionParams
from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

# Vertical: remote Streamable HTTP MCP server with explicit allowlist &
namespace prefix
erp_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(url="https://erp-
mcp.internal.example/mcp"),
    tool_filter=["get_order", "list_shipments"],  # Exact list or
ReadonlyContext predicate (never globs)
    tool_name_prefix="erp",
)

# Horizontal: cross-org or cross-language specialist discovered via A2A
Agent Card
customs_agent = RemoteA2aAgent(
    name="customs_clearance_agent",
    description="Validates international tariff codes and customs
holds.",
    agent_card="https://partner.example/.well-known/agent-card.json",
)
```

Integration layer	Production rules & Google Cloud integration (`PAA 3.2 / 3.3`)
MCP Transports & Lifecycle	Use **`StdioConnectionParams`** only for local dev subprocesses; use **`StreamableHTTPConnectionParams`** for remote stateless/stateful production services (legacy SSE-only transport is deprecated). Instantiate `McpToolset` **synchronously at module scope** in `agent.py` for `adk deploy`, and call `await toolset.close()` on custom FastAPI shutdown.
Google Cloud Managed MCP & Registry	Discover governed enterprise tools and agents through **Agent Registry** (`AgentRegistry`), **Cloud API Registry** (`ApiRegistry(mcp_server_name=...)` for Google Cloud MCP Servers like BigQuery, GKE, or Compute), and **MCP Toolbox for Databases** (`ToolboxToolset`). Authorize calls using **Agent Identity** (`principal://...`) and **Auth Manager** OAuth 2.0 tokens.
A2A Consumption & Publishing	Consume peers via `RemoteA2aAgent` (supports Go/Java/Python/LangGraph interoperability); expose an ADK agent via `to_a2a(agent, port=8001)` or `adk api_server --a2a`. Supports JSON-RPC 2.0 / gRPC / HTTP+JSON, SSE streaming (`message/stream`), and async webhook push notifications across task states (`submitted` → `working` → `input-required` / `auth-required` → `completed` / `failed` / `canceled` / `rejected`).
Adjacent Agentic Protocols	**A2UI:** declarative JSON UI payloads rendered only from the client's trusted component catalog. **AP2:** A2A payment extension enforcing cryptographically signed **Intent → Cart → Payment** mandates.

> **COMMON TRAP**
> Assuming local ADK `before_tool_callback` hooks inspect a `RemoteA2aAgent`'s internal database queries. Remote A2A agents are **opaque peers** executing outside your process—never expose internal prompts/tools over A2A, authenticate via the card's `securitySchemes`, and validate every returned A2A artifact at your boundary.

9. Skills, Skill Registry and Agents CLI

A **tool** executes an action; a **skill** packages domain procedural knowledge (`SKILL.md` instructions + reference docs + helper scripts) using **3-tier progressive disclosure** so a 50-skill agent doesn't burn 80,000 prompt tokens on turn 1.

Disclosure tier	Payload loaded into context	Triggered by ADK tool
L1: Routing metadata	YAML frontmatter `name` (kebab-case matching folder) + trigger-phrased `description` (~60 tokens)	`list_skills` / `search_skills` (or eager catalog summary)
L2: Playbook body	Complete `SKILL.md` workflow instructions (< 500 lines)	`load_skill`
L3: Deep assets & scripts	`references/` , `assets/` , and executable `scripts/`	`load_skill_resource` , `run_skill_script` (requires sandboxed `code_executor`)

- **Wiring local skills & `GCPSkillRegistry` :** Load local folders via `load_skill_from_dir(Path("./skills/incident-triage"))` into `SkillToolset(skills=[...], code_executor=AgentEngineSandboxCodeExecutor())` , or attach `GCPSkillRegistry` (**Agent Platform Skill Registry**) so agents dynamically discover versioned, organization-approved skills via `search_skills` without redeployment.
- **Configuring Skills & Plugins with `Agents CLI` (`PAA 2.2 / 3.1`):**
- **Skills & plugins management:** Install reusable skills (`npx skills add google/agents-cli` or `agents-cli` skill/plugin commands) and scaffold telemetry, Terraform, CI/CD, and security plugins across ADK workspaces.
- **Human mode vs. Agent mode (`PAA 3.1` exam focus):** In **human mode** (interactive TTY), `agents-cli` renders rich interactive wizards, colorized diffs, and confirmation prompts for developers. When invoked by autonomous coding harnesses (**Google Antigravity**, **Claude Code**, or CI runners), run `agents-cli` in **agent / non-interactive mode** (explicit CLI flags, no interactive stdin prompts, deterministic exit codes, and machine-readable JSON output) so automated workflows never hang waiting on terminal input. Same rule applies to L3 `SKILL.md` scripts: **non-interactive flags/env vars only**, `--help` support, and structured JSON on `stdout` .

> COMMON TRAP
> Treating an unreviewed third-party `SKILL.md` as harmless documentation. Skill instructions enter the model's privileged context and `run_skill_script` executes code—sync skills only from governed **Skill Registry** or code-reviewed repos, and never attach `UnsafeLocalCodeExecutor` to `SkillToolset` .

10. Callbacks, plugins and guardrails

Deterministic harness principle: Never rely on system-prompt wording (`"Please redact SSNs and never refund over $500"`) for security, compliance, or budget caps. Enforce hard invariants in deterministic ADK callbacks and `BasePlugin` s surrounding the model.

Lifecycle hook	Return `None`	Return a typed override object
`before_agent_callback(callback_context)`	Agent executes normally	`types.Content` → **skips agent run**; returns `Content` as the reply
`after_agent_callback(callback_context)`	Final output kept as-is	`types.Content` → **appends** an additional event (does **not** overwrite reply)
`before_model_callback(callback_context, llm_request)`	Calls LLM (`llm_request` mutable in place)	`LlmResponse` → **skips LLM call** (cache hit, fast policy block)
`after_model_callback(callback_context, llm_response)`	Keeps LLM output	`LlmResponse` → **replaces** model response (final output redaction/disclaimer)
`on_model_error_callback(callback_context, llm_request, error)`	Re-raises exception	`LlmResponse` → **suppresses error** and substitutes fallback (`after_model_callback` still runs)
`before_tool_callback(tool, args, tool_context)`	Executes tool (`args` mutable in place)	`dict` → **skips tool execution**; returns `dict` to the model as the tool result
`after_tool_callback(tool, args, tool_context, tool_response)`	Keeps raw tool result	`dict` → **replaces tool result** before it enters `session.events` or LLM context
`on_tool_error_callback(tool, args, tool_context, error)`	Re-raises tool exception	`dict` → **suppresses exception** and returns recovery `dict` (`after_tool_callback` still runs)

- **App-wide `BasePlugin` precedence:** Register cross-cutting guardrails (**Model Armor** screening, **Sensitive Data Protection** PII redaction, token budgets, audit logging) once on `App(name="...", root_agent=..., plugins=[...])` . **Plugin callbacks always execute before agent-level callbacks**, and the first hook returning a non-`None` override short-circuits remaining hooks in that phase.
- **Defense-in-depth ordering:** Screen user prompts in `before_model_callback` (fast regex → lightweight classifier / **Model Armor**) → validate tool parameters & spending caps in `before_tool_callback` → strip indirect prompt injections and **redact PII in `after_tool_callback`** → keep `after_model_callback` as the final outbound safety net.

> COMMON TRAP
> Scrubbing PII exclusively in `after_model_callback` . By that stage, unredacted customer records returned by a tool have already been committed to `session.events` , exported to traces (if content capture is on), and transmitted to the model context. Always scrub at **`after_tool_callback`** and **`before_tool_callback`** first.

11. Human-in-the-loop, resilience and durability

Operational requirement	ADK implementation pattern
Conditional approval on sensitive tools	`FunctionTool(fn, require_confirmation=lambda amount, **_: amount > 500)` (note: built-in `require_confirmation` is **`InMemorySessionService`** -only)
Production durable HITL across DB / Agent Runtime sessions	**`LongRunningFunctionTool`** (returns pending ticket/status immediately; external approval webhook resumes with matching `function_call.id`) or ADK 2.0 `Workflow` generator yielding `RequestInput(message=..., schema=...)`
Crash-resilient invocation replay	`App(..., resumability_config=ResumabilityConfig(is_resumable=True))` resumed with `invocation_id` (**at-least-once** execution—tools must be idempotent!)
Runaway reasoning loop protection (`PAA 4.2`)	`RunConfig(max_llm_calls=...)` (default **500**), `LoopAgent(max_iterations=...)` , plus per-session failure counters in `state`

3-Layer Fault Recovery Ladder: 1. **Transport layer (HTTP `429` / `503`):** Configure `HttpRetryOptions(attempts=3, initial_delay=1.0, exp_base=2.0)` on the `Gemini` model wrapper (and route stateless calls to the `global` endpoint or **Priority PayGo** / **Provisioned Throughput**). 2. **Tool argument self-correction:** Attach `ReflectAndRetryToolPlugin(max_retries=3, throw_exception_if_retry_exceeded=False)`. **Critical nuance:** out of the box it hooks raised exceptions (`on_tool_error_callback`); when your tools follow best practice and return `{"status": "error", ...}` dicts instead of raising, **override `extract_error_from_result`** on the plugin so structured error dicts trigger reflection. 3. **Model failover & graceful escalation:** Catch regional/model outages in `on_model_error_callback` (Python) or `RoutedLlm` (TypeScript) to fail over (e.g., `gemini-4-argon` → `gemini-3.8-flash`), and escalate deterministically when consecutive tool retries exhaust budget.

12. Context engineering, Gemini 4 Argon model routing and caching

Stuffing raw multi-turn transcripts and 50 KB JSON payloads into every specialist degrades accuracy ("lost in the middle"), inflates p95 latency, and multiplies token spend ~15× across multi-agent trees.

3-Tier Agentic Model Routing (`PAA 3.1`):

Routing tier	Recommended model & `thinking_level`	Workload & architectural role
Tier 1: High-QPS Router & Guardrail	**`gemini-3.5-flash-lite`** (`thinking_level="minimal"`) or fine-tuned **Gemma 4** SLM	Sub-200ms intent classification, slot extraction, schema normalization, and pre-LLM guardrail triage
Tier 2: Primary Agentic Workhorse	**`gemini-3.8-flash`** (`thinking_level="low"` or `"medium"`)	Default execution loop for MCP tool calling, code generation, RAG synthesis, and `AgentTool` specialists
Tier 3: Frontier Planner & Deep Synthesis	**`gemini-4-argon`** (`thinking_level="high"` or `"medium"`)	Ambiguous multi-step task decomposition, cross-document legal/financial conflict resolution, repo-scale refactoring, and unified 1M-token decodes
Real-Time Voice / Video Front-Door	**`gemini-3.8-live`** (`runner.run_live`) + **`gemini-4-argon`** (`AgentTool`)	`gemini-4-argon` does not support Live API WebSockets; handle sub-second voice + barge-in on `3.8-live` and delegate heavy backend reasoning via `AgentTool`

Context Compaction & Prefix Caching Mechanics (Gemini 4 Argon vs. Gemini 3.x):

Mechanism	Configured on	How it works & version-specific rules
`static_instruction` vs. `instruction`	`LlmAgent`	Keeps unchanging system rules, rubrics, and tool schemas at a byte-identical prefix (`static_instruction`) while appending dynamic `{state_key}` values afterward (`instruction`)
Implicit Context Caching (Gemini 4 Argon & 3.x)	Automatic (model API)	**`gemini-4-argon` uses implicit caching exclusively** (triggers automatically on shared prefixes `≥ 4,096` tokens with zero hourly storage fee; **explicit `cachedContents` creation fails on `gemini-4-argon`**). Prefix stability via `static_instruction` is essential!
`ContextCacheConfig(cache_intervals, ttl_seconds, min_tokens)`	`App`	Manages explicit TTL `cachedContents` handles on **Gemini 3.x / 2.5** base models when deterministic cache retention across sparse traffic justifies storage fees
`EventsCompactionConfig(compaction_interval, overlap_size)`	`App`	Periodically summarizes older `session.events` slices into compact summary events so long-running sessions don't hit context degradation
`include_contents='none'` + `{state_key}`	`LlmAgent`	Strips prior chat history completely for stateless extractors, formatters, and independent auditors
Observation masking (`after_tool_callback`)	`LlmAgent` / `BasePlugin`	Trims verbose 40-column API responses down to the 4 needed fields before the event enters `session.events`

> **COMMON TRAP**
> Trying to provision explicit `cachedContents` (`ContextCacheConfig`) for **`gemini-4-argon`**, or injecting `{current_timestamp}` on line 1 of `instruction`. `gemini-4-argon` rejects explicit `cachedContents` and relies on **automatic implicit caching (`≥ 4,096` tokens)**—which drops to a 0% hit rate if a dynamic timestamp mutates the start of the prompt!

13. Security and sandboxing

Threat vector	Attack scenario	Deterministic ADK control
Direct prompt injection	User attempts to override system instructions	`before_model_callback` + **Vertex AI Model Armor** prompt screening
Indirect prompt injection	Malicious instructions hidden inside fetched web pages, emails, or PDFs	Sanitize/screen untrusted payloads in `after_tool_callback`; isolate untrusted reader agents from write-capable tools
Confused deputy / excessive agency	Read-only support bot tricked into invoking `delete_account`	Least-privilege `tool_filter` allowlist, `WriteMode.BLOCKED`, and per-agent **Agent Identity**
Cross-agent taint propagation	Compromised sub-agent poisons downstream publisher	Enforce strict Pydantic `output_schema` + boundary validation callbacks between handoffs
Host container escape via generated code	LLM-generated Python reads `/proc/self/environ` or metadata credentials	Never use `UnsafeLocalCodeExecutor` in prod; isolate in **Agent Runtime Code Execution** or **GKE Agent Sandbox (gVisor)**

`code_executor` implementation	**Isolation boundary & statefulness**	**Recommended environment**
`BuiltInCodeExecutor`	Gemini server-side sandbox (stateless, 30 s max, zero network egress; cannot mix directly with custom tools on one agent)	Zero-infra math, data wrangling, charts
`AgentEngineSandboxCodeExecutor`	Managed **Agent Runtime** isolated container (stateful files across turns, up to 14-day TTL, 100 MB I/O)	Default production choice on Google Cloud
`GkeCodeExecutor(executor_type="sandbox")`	Pre-warmed **GKE Agent Sandbox** backed by **gVisor** kernel isolation (sub-second startup; replaces slow per-run K8s `"job"` mode)	GKE clusters needing custom VPC/hardware control

`code_executor` implementation	Isolation boundary & statefulness	Recommended environment
`ContainerCodeExecutor`	Dedicated Docker container	Local staging or custom system binary testing
`UnsafeLocalCodeExecutor`	Runs `exec()` inside the ADK host process with full IAM/env access	Throwaway local debugging only—**never** staging or prod

Three identity authorities (`PAA 3.2 / 5.1`): (1) Google Cloud Agent Identity (`principal://...` / `spiffe://...` per-agent cryptographic identity bound to **Principal Access Boundary [PAB]** policies and **Dual-Attribution Cloud Audit Logs**), **(2) Workload Service Account** (coarse infrastructure identity), and **(3) User OAuth 2.0 Delegation** (`Tool Context.request_credential` / **Auth Manager** when acting strictly under the end user's own entitlements).

14. Evaluation: ADK `evalset` vs. Gen AI Evaluation Service & observability

Core principle (`PAA 4.1`): Grade **both** the final response and the **tool trajectory**. An agent that guesses the right refund figure while skipping mandatory `verify_return_window` compliance checks is a production defect.

Architectural Decision Matrix: Which Evaluation Engine Should You Use? (`PAA 4.1`)

Evaluation tier	Primary tool / API	Executes where & against what?	Key strengths & when to choose
1. Deterministic Harness Replay	`adk conformance record` + `adk conformance test --mode=replay`	Local / PR CI in `<5s` using recorded LLM responses (zero live model calls)	Verifies Python/Go/TS callback wiring, `state_delta` routing, and guardrail refactors with **zero token cost and zero sampling variance** on every commit
2. Local & CI Golden Regression	`adk eval <dir> <evalset.json>` / `pytest` `AgentEvaluator.evaluate(...)` / `adk web` Eval tab	In-repo CI runner executing your ADK agent against `*.test.json` (unit) and `*.evalset.json` (multi-turn)	Fast developer inner-loop and PR merge gate grading `tool_trajectory_avg_score` (`EXACT`, `IN_ORDER`, `ANY_ORDER`), `final_response_match_v2`, `hallucinations_v1`, `UserSimulatorProvider`, and Python `custom_metrics`
3. Managed Cloud Fleet Evaluation & Simulation	**Agent Platform Gen AI Evaluation Service** (`client.evals` / `EvalTask` / `agents-cli eval`)	Managed Google Cloud service grading **any** agent (ADK on Agent Runtime, Go/TS/Java Cloud Run endpoints, LangGraph, or custom callables)	**Choose when you need: Adaptive Rubrics** (auto-generated per-prompt pass/fail checks), custom **Autoraters**, **2-Step Agent Simulation** (from immutable **Agent Registry** snapshots), **Environment Simulation** (injecting HTTP 503/429/timeout tool faults safely), **Online Evaluation Monitors** on live Cloud Trace spans, **Failure Cluster Analysis**, or closed-loop **Agent Optimizer** prompt tuning

ADK `test_config.json` criterion	What it grades	Rater type & exam nuances
`tool_trajectory_avg_score`	Predicted vs. reference tool sequence (`match_type`: `EXACT` default, `IN_ORDER` allows extra helper calls between required steps, `ANY_ORDER` for parallel lookups)	Deterministic (mirrors cloud `trajectory_exact_match` / `trajectory_in_order_match` / `trajectory_any_order_match` / `trajectory_precision` / `trajectory_recall`)
`response_match_score` vs. `final_response_match_v2`	Lexical ROUGE-1 overlap vs. semantic equivalence	Deterministic vs. LLM judge (prefer `final_response_match_v2` whenever valid phrasing varies)
`rubric_based_final_response_quality_v1` / `rubric_based_tool_use_quality_v1`	Domain-specific quality rubrics	LLM judge
`hallucinations_v1` & `safety_v1`	Context faithfulness (catches invented numbers despite calling the right tool) & harmlessness	LLM judge (`safety_v1` delegates to Gen AI Eval Service)
`multi_turn_task_success_v1` / `multi_turn_trajectory_quality_v1`	End-to-end goal completion with `UserSimulatorProvider` (`starting_prompt` + `conversation_scenario`)	LLM judge
`tool_call_count_v1`, `token_usage_v1`, `invocation_duration_v1`	Resource efficiency telemetry	`INFORMATIONAL` only—setting thresholds on these in `test_config.json` throws an error; enforce hard call/cost gates via Python `custom_metrics` (`code_config.name`)

Observability & Troubleshooting (`PAA 4.2`): - **OpenTelemetry GenAI spans:** ADK emits hierarchical spans (`invoke_agent` → `call_llm` → `execute_tool`). Export via `--otel_to_cloud` locally or `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true` on Agent Runtime (powers the 8 **Agent Registry Observability** tabs: Overview, Evaluation, Models, Tools, Usage, Logs, Traces, Topology). - **Privacy-by-default:** Keep `ADK_CAPTURE_MESSAGE_CONTENT_IN_SPANS=false` (and `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT` unset/false) in production unless paired with **Sensitive Data Protection** de-identification and restricted log buckets. - **Production flywheel:** Export low-scoring **Online Evaluation Monitor** traces and human escalations → scrub PII with SDP → convert into permanent `*.evalset.json` / golden simulation cases so fixed regressions never recur.

> COMMON TRAP
> Testing agent resilience against rate limits or ERP outages by firing tests against live staging/production APIs, or relying solely on local `*.test.json` files for multi-language cloud fleets. Use **Environment Simulation** + **2-Step Agent Simulation** in the **Gen AI Evaluation Service** against an immutable **Agent Registry** snapshot to inject deterministic 503/429 faults safely.

15. Deployment, live streaming and developer tooling

Deployment target (`PAA 4.2`)	CLI workflow	Infrastructure you manage	Choose when
Agent Runtime (managed `reasoning Engines`)	`adk deploy agent_engine` or `agents-cli deploy`	Agent code & dependencies only (**Python** payload omits API server; **Go** payload bundles its dedicated ADK API server)	**Default on Google Cloud:** turnkey `VertexAi Session Service` , **Memory Bank**, **Code Execution** sandbox, **Agent Identity**, and **Gemini Enterprise** registration
Cloud Run	`adk deploy cloud_run [--trigger_ sources="pubsub,eventarc"]`	Container image, session DB (`Database Session Service` / `agentengine://`), scaling	**TypeScript / Java / Kotlin** ADK containers, custom FastAPI/HTTP gateways, scale-to-zero, or **ambient event-driven agents** triggered by Pub/Sub & Eventarc
GKE	`adk deploy gke` / custom Helm	Cluster, node pools, networking	Strict VPC/data-residency isolation, co-located self-hosted **Gemma 4** GPUs/TPUs, or **gVisor Agent Sandbox** (`GkeCode Executor`)

PLATFORM DEEP DIVE
See the Agent Platform guide for capacity planning, regional compute exhaustion vs. project quotas, Provisioned Throughput GSUs, and Agent Registry governance.

```
import asyncio
from google.genai import types
from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

agent = LlmAgent(
    name="architect_helper",
    model="gemini-4-argon",

planner=BuiltInPlanner(thinking_config=types.ThinkingConfig(include_thoug
hts=True, thinking_level="medium")),
    instruction="Answer concisely with concrete architectural trade-
offs.",
)

async def run_turn(prompt: str) → str:
    session_service = InMemorySessionService()  # Replace with
VertexAiSessionService / DatabaseSessionService in prod
    session = await session_service.create_session(app_name="hub",
user_id="u_42")
    runner = Runner(agent=agent, app_name="hub",
session_service=session_service)
    msg = types.Content(role="user", parts=[types.Part(text=prompt)])
    final_answer = ""
    async for event in runner.run_async(user_id="u_42",
session_id=session.id, new_message=msg):
        if event.is_final_response() and event.content and
event.content.parts:
            # Drain ALL events in multi-agent runs and filter out
internal thought summaries (p.thought == True)
            final_answer = "".join(p.text or "" for p in
event.content.parts if not getattr(p, "thought", False))
    return final_answer

if __name__ == "__main__":
    print(asyncio.run(run_turn("When should I choose AgentTool over
sub_agents?")))
```

- **Real-time bidirectional voice & video (`runner.run_live`):** Open low-latency **Gemini 3.8 Live (`gemini-3.8-live`)** sessions via `runner.run_live(user_id=..., session_id=..., live_request_queue=LiveRequestQueue(), run_config=RunConfig(response_modalities=["AUDIO"]))` (`send_realtime(Blob)` for PCM audio/video frames, halt playback immediately when `event.interrupted == True` on user barge-in). Remember: setting `StreamingMode.BIDI` on `runner.run_async()` is a **silent no-op** in Python ADK (`run_async` only supports `Streaming Mode.SSE`), and **`gemini-4-argon` does not support Live API WebSockets**—pair `gemini-3.8-live` at the voice front-door with a `gemini-4-argon` `AgentTool` for deep backend reasoning.

16. Failure modes and exam traps

Symptom or production failure	Root cause → architectural fix	Tempting distractor (wrong)
`HTTP 400 INVALID_ARGUMENT` when configuring thinking on `gemini-4-argon` or `gemini-3.8-flash`	Passed `thinking_level="minimal"` (unsupported on `4-argon` & `3.8-flash`) or legacy integer `thinking_budget` on `4-argon` → use `thinking_level` in `{"low", "medium", "high"}` (reserve `"minimal"` for `gemini-3.5-flash-lite`)	Setting `temperature=0.0` to disable thinking
Explicit `cached Contents` creation fails when migrating an ADK app to `gemini-4-argon`	`gemini-4-argon` supports **implicit caching only** (⩾ `4,096` tokens, zero storage fee) → remove explicit cache creation and place static prompt content in `static_instruction`	Increasing `ContextCache Config.ttl_seconds`

Symptom or production failure	Root cause → architectural fix	Tempting distractor (wrong)
`ManagedAgent` raises `NotImplemented Error` or regional connection error	Attached local Python `Function Tool` / `McpToolset` or targeted `us-central1` → `ManagedAgent` supports **server-side tools only** and requires the **`global`** location on Agent Platform; keep local/MCP tools on a companion `LlmAgent`	Wrapping `Mcp Toolset` in `StdioConnection Params`
Chaining `previous_ interaction_id` on `Interactions API` (`background=True`) returns `400 Bad Request`	Previous background interaction is still `in_progress` → poll interaction status until `completed` before sending the follow-up turn	Resending the full chat history without `previous_ interaction_id`
Java/Kotlin ADK team tries to import ADK 2.0 `Workflow(edges=[...])` or mix `Google SearchTool` + custom tools on one agent	ADK 2.0 graph `Workflow` is currently **Python, TypeScript, and Go**; Java/Kotlin use ADK 1.x workflow agents (`Sequential Agent` / `ParallelAgent` / `Loop Agent`) and must isolate `GoogleSearchTool` inside an `AgentTool` specialist	Enabling `bypass_ multi_tools_ limit=True` in Java
Coding agent hangs indefinitely running `agents-cli` or a `SKILL.md` helper script in CI	Executed in interactive human TTY mode waiting on stdin confirmation → invoke `agents-cli` and skill scripts in **non-interactive agent mode** with explicit CLI flags and JSON stdout	Extending the Cloud Build timeout to 60 minutes
Cross-user data leak under load	State stored on `self` inside a singleton `Agent` → move all mutable state to `Session Service` (`app_name` / `user_ id` / `session_id`)	Instantiating a new MCP server per HTTP request
`LoopAgent(max_ iterations=3)` finishes without converging, yet downstream `publisher` still runs	`LoopAgent` exits **silently** at `max_iterations` → set `state["temp:approved"] = True` in the exit tool alongside `escalate = True` and gate `publisher` via `before_ agent_callback`	Catching `Timeout Error` around `LoopAgent`
Specialist hijacks user conversation instead of returning a structured sub-result	Placed specialist in `sub_agents` (`mode="chat"` transfer) → wrap in `AgentTool(agent=...)` or use ADK 2.0 `mode="single_ turn"`	Prompting `"Do not talk to the user"`

Symptom or production failure	Root cause → architectural fix	Tempting distractor (wrong)
`KeyError` before LLM call when instruction contains `{"id": 1}`	ADK parses `{id}` as a `session.state` placeholder → pass an `InstructionProvider` callable `def prompt(ctx: ReadonlyContext) -> str:`	Escaping braces as `{{id}}`
Early `return` on `event.is_final_ response()` returns step 1's draft or raw thoughts	Every sub-agent in a pipeline emits `is_final_response()` → drain the full `run_async` generator and filter `not getattr(p, "thought", False)`	Switching from `run_async` to `run_live`
`ReflectAndRetryTool Plugin` ignores failing tools	Tools return `{"status": "error"}` dicts instead of raising exceptions → override `extract_error_from_result` on the plugin	Raising `max_ retries`
Built-in `require_ confirmation=True` breaks after switching to `DatabaseSession Service` / `VertexAi SessionService`	Built-in tool confirmation is `In MemorySessionService` -only → use `LongRunningFunction Tool` or ADK 2.0 `Request Input`	Switching SQLAlchemy drivers
Raw SSNs appear in `session.events` and traces despite `after_ model_callback` redaction	Tool results are persisted and sent to the LLM *before* `after_ model_callback` → scrub PII at the tool boundary (`after_tool_ callback` & `before_tool_ callback`) and keep `ADK_ CAPTURE_MESSAGE_CONTENT_ IN_SPANS=false`	Strengthening system prompt privacy rules
`adk eval` fails when setting a threshold on `tool_call_count_v1` or `token_usage_v1`	Built-in efficiency metrics are `INFORMATIONAL` only → enforce call/token budgets via Python `custom_metrics` (`code_ config.name`)	Lowering threshold values to `0.5`
Team wants to test 503 tool outages and auto-generate multi-turn personas across a deployed Go Cloud Run agent	`adk eval` is designed for local/CI golden files → use **Agent Platform Gen AI Evaluation Service** (**2-Step Agent Simulation** + **Environment Simulation** + **Adaptive Rubrics**)	Running chaos faults against live production ERP APIs